

浙江大学 实验报告

姓名: 马玉玺

学号: 3240104676

日期: 2025 年 8 月 24 日

实验名称:

TinyLLM

一、实验思路

1. `modules/mlp.py` 中 `Qwen3FFN` 类的实现思路

`Qwen3FFN` 模块的核心是实现一个采用 SwiGLU 激活函数的 FFN。与传统的 FFN 不同,SwiGLU 引入了门控机制,这有助于模型更好地控制信息流,从而提升性能。我的实现思路是精确地遵循 SwiGLU 的数学定义:
`down_proj(SiLU(gate_proj(x)) * up_proj(x))`。

前向传播过程非常直观:

输入 `x` 被同时送入 `gate_proj` 和 `up_proj`。`gate_proj` 的输出通过一个 SiLU 激活函数。将 SiLU 的输出与 `up_proj` 的输出进行逐元素相乘。这一步是门控机制的核心,通过门控值来动态调节信息。最后,将相乘的结果通过 `down_proj` 投影回最终的输出维度。

2. `modules/attention.py` 中 `Qwen3Attention` 类的实现思路

我的实现思路是将整个前向过程分解为一系列清晰的步骤,并将 GQA、RoPE 和 Q/K Normalization 等技术集成进去。实现思路步骤如下:

- QKV 投影与 GQA 的体现: 输入 `hidden_states` 首先通过三个独立的线性层 `q_proj`、`k_proj` 和 `v_proj` 来生成 Query、Key 和 Value。为了实现 GQA,`k_proj` 和 `v_proj` 的输出维度被设置为 `num_key_value_heads * head_dim`, 这个维度小于 `q_proj` 的 `num_attention_heads * head_dim`。这直接从硬件层面减少了 KV Cache 的大小和计算量。
- 多头格式变换: 将投影后的 Q, K, V 张量通过 `view` 和 `transpose` 操作重塑为多头格式,即 `[batch_size, num_heads, seq_len, head_dim]`, 为并行的注意力计算做准备。
- Q/K Normalization: 在应用位置编码和计算注意力分数之前, 对重塑后的 `query_states` 和 `key_states` 在 `head_dim` 维度上应用 `RMSNorm`。这有助于稳定训练过程和提高模型性能。
- RoPE 位置编码: 通过 `rotary_emb` 模块计算出对应序列长度的旋转矩阵 (`cos` 和 `sin`), 然后调用 `apply_rotary_pos_emb` 函数将位置信息注入到归一化后的 Q 和 K 中。这种方式比传统的位置编码能更好地捕捉相对位置关系。
- GQA 的广播机制: 由于 K 和 V 的头数少于 Q,需要通过 `_repeat_kv` 方法将 K 和 V 的头进行复制,使其数量与 Q 的头数相匹配,从而能够进行后续的点积注意力计算。

6. 标准的缩放点积注意力：执行标准的注意力计算流程:计算 Q 和 K 的点积、进行缩放、应用因果掩码、通过 `softmax` 获得注意力权重,最后将权重与 V 相乘得到输出。
7. 输出投影： 将多头注意力的输出结果重新合并，并通过最后的线性层 `o_proj` 将其投影回 `hidden_size` 维度，得到最终的注意力输出。

二.单元测试通过的截图和模型生成效果的截图

```
~ — h3240104676@sct101: ~/hpc101/src/lab5 — ssh hpc101 +
生成时间: 2.33 秒
生成 token 数量: 10
生成速度: 4.29 tokens/秒
=====
完整输出: To be or not to be... a good student? That is the question.
基线模型测试完成

=====
                        测试结果对比
=====

测试点 1: ✔ 通过
提示词: Once upon a time in a land far, far away, there lived a dragon who
目标模型: Once upon a time in a land far, far away, there lived a dragon who had a peculiar habit of hoarding not gold or
基线模型: Once upon a time in a land far, far away, there lived a dragon who had a peculiar habit of hoarding not gold or

测试点 2: ✔ 通过
提示词: The quick brown fox
目标模型: The quick brown fox jumps over the lazy dog. This is a pang
基线模型: The quick brown fox jumps over the lazy dog. This is a pang

测试点 3: ✔ 通过
提示词: 在一个阳光明媚的早晨
目标模型: 在一个阳光明媚的早晨, 我决定去附近的公园散步。公园里
基线模型: 在一个阳光明媚的早晨, 我决定去附近的公园散步。公园里

测试点 4: ✔ 通过
提示词: To be or not to be
目标模型: To be or not to be... a good student? That is the question.
基线模型: To be or not to be... a good student? That is the question.

所有测试完成

=====
                        最终测试汇总
=====
一致性测试结果: 4/4 通过
🎉 恭喜! 所有测试都通过了! 模型实现正确!
=====
单元测试完成。
开始运行模型生成 (main.py)...
```

Figure 1: 单元测试通过的截图

```
h3240104676@sct101: ~/hpc101/src/lab5 — ssh hpc101
目标模型: The quick brown fox jumps over the lazy dog. This is a pang
基线模型: The quick brown fox jumps over the lazy dog. This is a pang

测试点 3: ✔ 通过
提示词: 在一个阳光明媚的早晨
目标模型: 在一个阳光明媚的早晨, 我决定去附近的公园散步。公园里
基线模型: 在一个阳光明媚的早晨, 我决定去附近的公园散步。公园里

测试点 4: ✔ 通过
提示词: To be or not to be
目标模型: To be or not to be... a good student? That is the question.
基线模型: To be or not to be... a good student? That is the question.

所有测试完成

=====
                    最终测试汇总
=====
一致性测试结果: 4/4 通过
🎉 恭喜! 所有测试都通过了! 模型实现正确!
=====
单元测试完成。
开始运行模型生成 (main.py)...
=====

输入提示: 为什么西瓜比苹果甜? <think>
=====
输入 token 数量: 7

生成完成!
生成时间: 62.69 秒
生成 token 数量: 200
生成速度: 3.19 tokens/秒
=====
完整输出: 为什么西瓜比苹果甜? <think>
嗯, 用户问为什么西瓜比苹果甜, 这个问题看起来简单, 但其实可能需要从多个角度来分析。首先, 我得回忆一下自己学过的植物学和营养学知识。西瓜和苹果都是常见的水果, 但它们的甜度确实不同。可能的原因包括糖分种类、糖分含量、水分含量、口感因素, 还有可能涉及到植物的进化适应性。

首先, 糖分种类。西瓜主要含有葡萄糖和果糖, 而苹果可能含有更多的蔗糖? 或者是不是相反? 我记得蔗糖的甜度比葡萄糖和果糖高, 但可能西瓜的糖分比例不同。不过, 可能需要查证一下具体的糖分组成。比如, 西瓜的糖分主要是葡萄糖和果糖, 而苹果的糖分可能更多是蔗糖? 或者是不是相反? 或者可能两者都有, 但比例不同?

然后是糖分含量。西瓜的糖分含量可能更高? 比如, 西瓜的含
```

Figure 2: 模型生成效果的截图

三、思考题

3.1. Qwen3 Decoder Layer 的结构和前向过程

Qwen3 Decoder Layer 遵循一个标准的“Pre-Norm”Transformer 结构, 其核心思想是在将输入送入主要计算模块之前进行 Layer Normalization。该结构主要由以下四个组件构成:

self_attn: 负责计算序列中各个 Token 之间的注意力权重, 并生成加权的上下文表示。

mlp: 采用 SwiGLU 激活函数, 对自注意力模块的输出进行非线性变换, 以增强模型的表达能力。

input_layernorm: 在自注意力模块之前对输入进行归一化。

post_attention_layernorm: 在前馈网络模块之前, 对自注意力模块的输出 (已加上残差连接) 进行归一化。

此外, 该结构包含两个关键的残差连接, 分别位于自注意力模块和前馈网络模块之后, 用于缓解梯度消失问题, 并使得模型能够学习恒等映射。

前向过程 假设输入张量为 `hidden_states`, 其形状为 $[B, S, H]$, 其中 B 是批量大小 (batch_size), S 是序列长度 (seq_len), H 是隐藏层维度 (hidden_size)。前向传播过程如下:

第一个残差分支 (Pre-Attention): a. 首先, 保存一份原始的 `hidden_states` 作为第一个残差连接 residual。 b. 将 `hidden_states` 送入 `input_layernorm` 进行

RMS 归一化。 c. 将归一化后的张量送入 `self_attn` 模块, 得到注意力输出 `attn_output`。 d. 将 `attn_output` 与第一步保存的 `residual` 相加, 完成第一个残差连接: `hidden_states = residual + attn_output`。

第二个残差分支 (Pre-FFN): a. 保存上一步的结果作为第二个残差连接 `residual`。 b. 将该结果送入 `post_attention_layernorm` 进行 RMS 归一化。 c. 将归一化后的张量送入 `mlp` 模块, 得到前馈网络输出 `ffn_output`。 d. 将 `ffn_output` 与第二步保存的 `residual` 相加, 完成第二个残差连接: `hidden_states = residual + ffn_output`。

输出: 返回经过两个完整分支处理后的最终 `hidden_states` 张量, 其形状仍为 $[B, S, H]$ 。

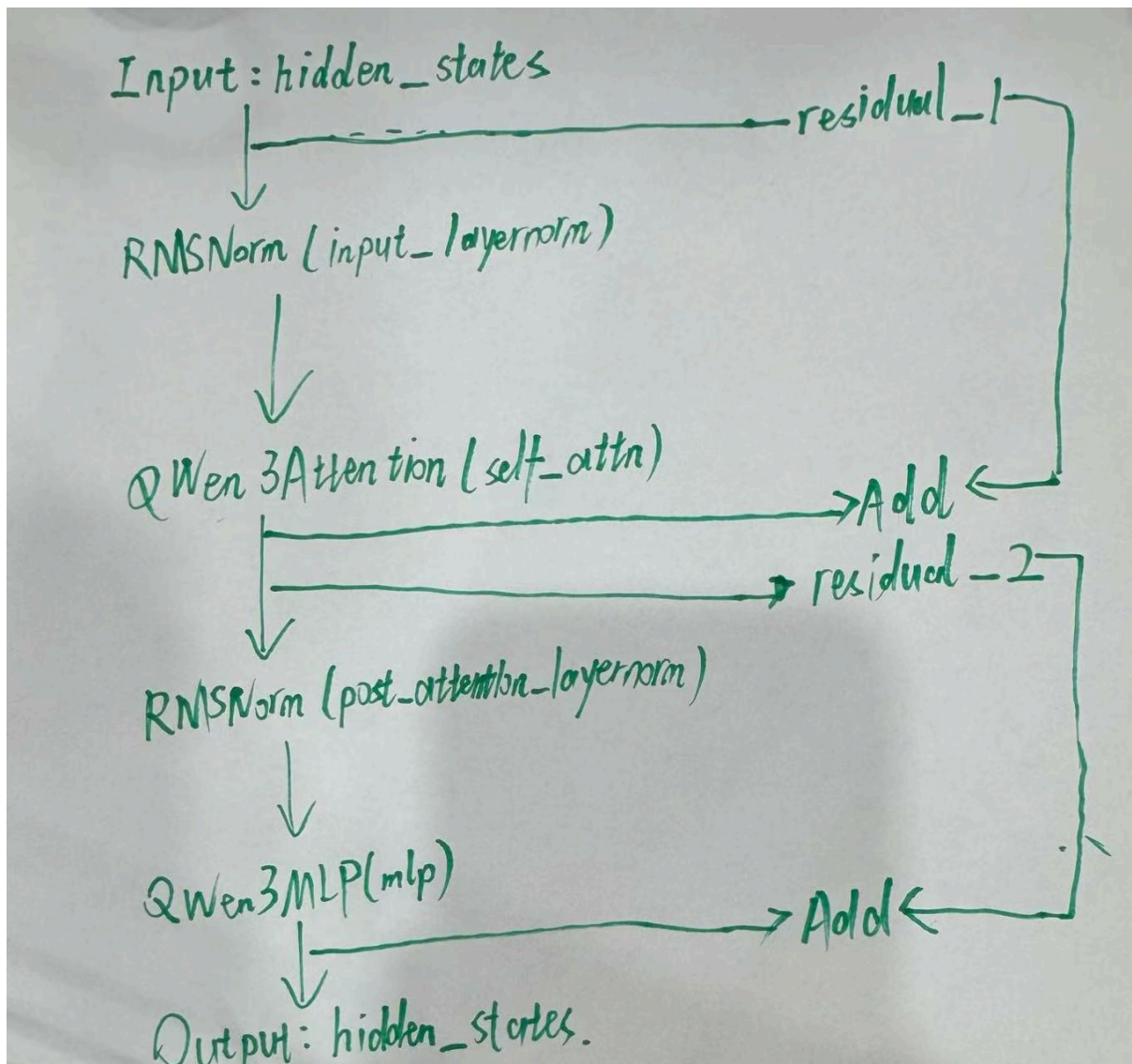


Figure 3: 画图说明

3.2. Qwen3 模型显存占用情况分析

参数量计算 Token Embedding: $\text{vocab_size} \times \text{hidden_size} = 151936 \times 4096 = 622,330,024$

单个 Decoder Layer:

Attention 模块:

q_proj: $\text{hidden_size} * \text{num_attention_heads} * \text{head_dim} = 4096 * 4096 = 16,777,216$

k_proj: $\text{hidden_size} * \text{num_key_value_heads} * \text{head_dim} = 4096 * 8 * 128 = 4,194,304$

v_proj: $\text{hidden_size} * \text{num_key_value_heads} * \text{head_dim} = 4096 * 8 * 128 = 4,194,304$

o_proj: $\text{num_attention_heads} * \text{head_dim} * \text{hidden_size} = 4096 * 4096 = 16,777,216$

q_norm + k_norm: $\text{head_dim} + \text{head_dim} = 128 + 128 = 256$

MLP 模块:

gate_proj: $\text{hidden_size} * \text{intermediate_size} = 4096 * 12288 = 50,331,648$

up_proj: $\text{hidden_size} * \text{intermediate_size} = 4096 * 12288 = 50,331,648$

down_proj: $\text{intermediate_size} * \text{hidden_size} = 12288 * 4096 = 50,331,648$

LayerNorm 模块:

input_layernorm: $\text{hidden_size} = 4096$

post_attention_layernorm: $\text{hidden_size} = 4096$

单层总计: $(16.78\text{M} + 4.19\text{M} + 4.19\text{M} + 16.78\text{M}) + 3 * 50.33\text{M} + 2 * 4096 + 256 \approx 192,943,616$

所有 Decoder Layers 总计: $\text{num_hidden_layers} * 36 * 192,943,616 = 6,945,970,176$

Final RMSNorm: $\text{hidden_size} = 4096$

LM Head : $\text{hidden_size} * \text{vocab_size} = 4096 * 151936 = 622,330,024$

模型总参数量: $622.3\text{M (Embed)} + 6946.0\text{M (Layers)} + 4\text{K (Norm)} + 622.3\text{M (LM Head)} \approx 8,190,634,328$ (8.19 Billion)

在本实验中，模型使用 bfloat16 数据类型，每个参数占用 2 字节。

模型权重显存占用: $\text{总参数量} * 2 \text{ bytes} = 8,190,634,328 * 2 \text{ bytes} = 16,381,268,656 \text{ bytes}$
 $16,381,268,656 / 1024^3 \approx 15.26 \text{ GB}$

```
开始模型一致性测试...
GPU 初始显存使用: 0.00 MiB
=====
正在加载目标模型...
加载目标模型后 GPU 显存使用: 17064.62 MiB
目标模型占用显存: 17064.62 MiB
目标模型生成中...
```

Figure 4: 显存占用

原因: PyTorch 和 NVIDIA 驱动在初始化时会在 GPU 上创建一个 CUDA 上下文, 用于管理内核启动、内存分配等。这本身就会占用一部分显存。此外, PyTorch 还会预分配一部分显存以优化内存管理和分配效率, 这也是为什么实际可用显存会少于标称值的原因之一。